

MATCH 3 - TECHNICAL DESIGN DOCUMENTATION

Match-3 games belong to a broader category of **state-based deterministic games**. In these games, the entire game can be described as a sequence of discrete states, where each new state is produced by applying deterministic rules to a previous state.

Well-known examples of state-based games include Chess, Tetris, Go, Checkers, and most card games. Unlike character-driven games that rely heavily on continuous input and physics, state-based games depend on **predictable state transitions** and **clear resolution phases**. This naturally pushes the architecture toward determinism, clarity, and strict separation of responsibilities.

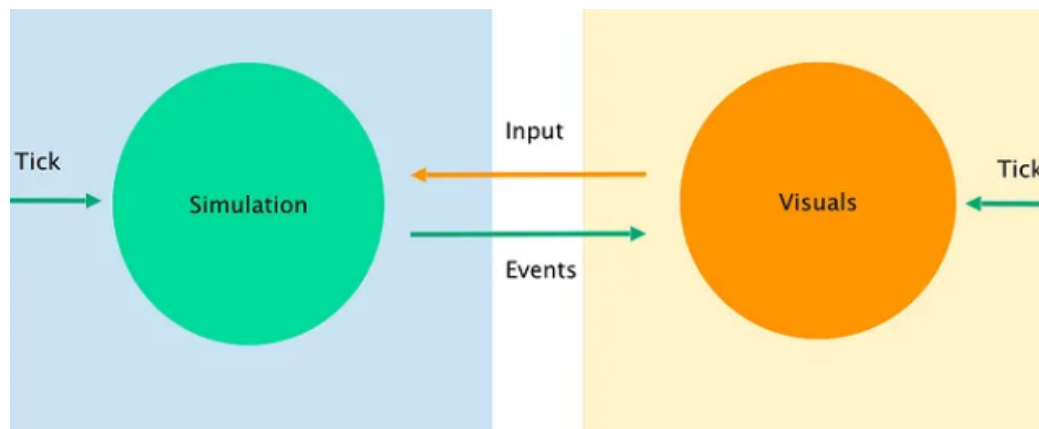
Common Architecture for State-Based Games

A typical architecture separates the system into two major parts:

- **Simulation (Engine)**
The core game logic that operates purely on data.
Examples: match-3 engine, chess engine, Tetris engine.
- **Visuals (Frontend)**
The game-engine-specific layer (Unity in this case) responsible only for:
 - Input collection
 - Rendering
 - Animation
 - Audio

The simulation layer is usually **game-engine agnostic** and processes input to produce new game states. The visuals listen to changes and reflect them on screen.

In many implementations, this communication is **event-based**:



This approach provides loose coupling, but it introduces several well-known drawbacks:

- Harder unit testing (logic tied to event order)
- Debugging complexity (state inferred from event sequences)
- High risk of desynchronization
- Strict lifecycle and ordering requirements

While viable, this approach scales poorly as systems grow more complex (bombs, cascades, delayed effects, chain reactions).

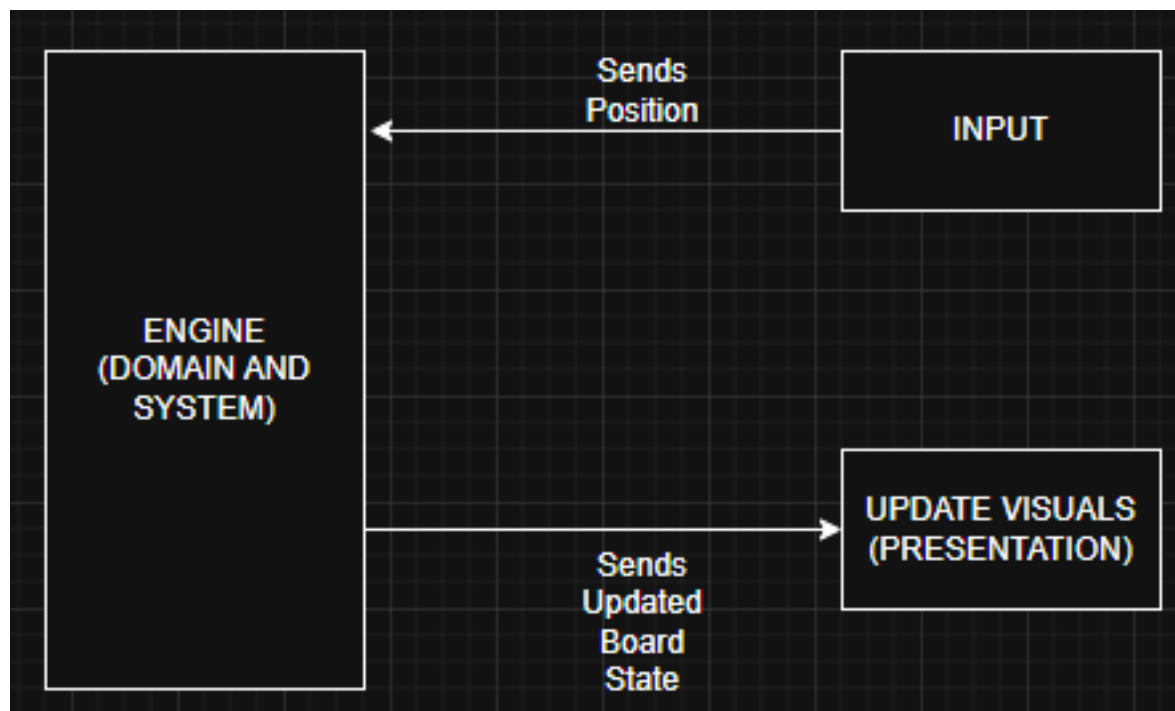
Chosen Architecture: Data-Driven State Resolution

For this match-3 engine, we deliberately chose a **data-driven state transition model** instead of an event-driven one.

This design emphasizes:

- Determinism
- Testability
- Debuggability
- Scalability
- Minimal coupling with the rendering layer

Rather than emitting events during resolution, the engine operates entirely on immutable or controlled board states.



Input Model

Unity provides input to the engine in the simplest possible form:

- Two board positions representing a swap attempt

No visual or animation logic exists inside the engine. The engine receives **pure data**, processes it, and produces **pure data** in return.

State Resolution Flow

The engine performs the following steps:

1. **Snapshot the initial board state**
2. Apply the requested mutation (e.g., gem swap)
3. Resolve all consequences in a deterministic loop:
 - Match detection
 - Match clearing
 - Bomb resolution
 - Gravity resolution
 - New gem generation (if applicable)
4. Repeat resolution until the board reaches a **stable state**
 - No matches
 - No pending effects
5. **Snapshot the final board state**

At the end of this process, the engine returns:

- The **initial board state**
- The **final resolved board state**

No intermediate animation steps are required by the engine.

Presentation Responsibility

Unity receives both board states and becomes responsible for:

- Diffing before/after states
- Animating swaps, falls, explosions, and cascades
- Timing and visual polish

This makes the presentation layer entirely declarative:

“Animate how we got from State A to State B”

The engine never needs to know *how* something is animated—only *what changed*.

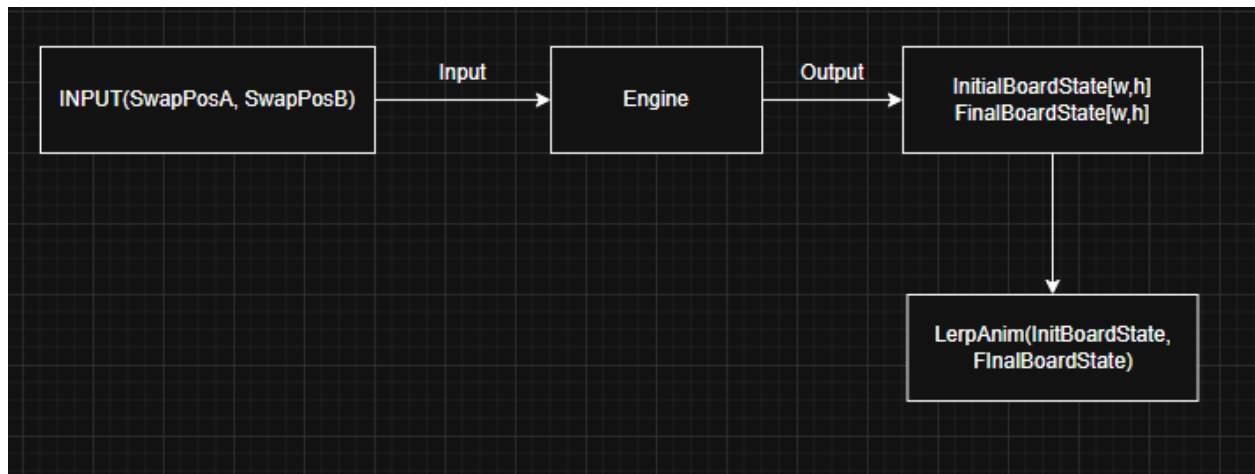
Benefits of This Approach

This architecture provides several concrete advantages:

- **Highly testable**
The entire game logic can be tested without Unity or any rendering system.
- **Deterministic and debuggable**
Bugs can be reproduced by replaying a single input against a known board state.
- **No desync risk**
Visuals always reflect a fully resolved, authoritative state.
- **Scalable resolution logic**
Complex mechanics (bombs, chains, delays) can be added without increasing event complexity.
- **Loose coupling by design**
The engine does not depend on Unity, coroutines, animations, or frame timing.

High-Level Summary

This design treats the match-3 game as a **pure state machine**:



Unity acts purely as an interpreter of state changes, not as a participant in the game logic.

This approach aligns naturally with state-based games and provides a clean, maintainable foundation for long-term development.

CodeArchitectural Overview

Domain, System, and Service Separation

This project follows a clear separation between **domain**, **system**, and **service** layers to ensure determinism, testability, and long-term maintainability.

Domain

The **domain layer** represents the *rules and state of the game*.

It is:

- Pure
- Deterministic
- Free of orchestration concerns
- Free of engine or presentation dependencies

Examples:

- Board state
- Gem state
- Match definitions
- Gravity results

Domain objects describe **what a game entity is**, not **how it runs**.

Logic

Logics execute domain rules in a deterministic and repeatable way.

They:

- Take domain data as input
- Apply a single category of rules
- Produce new domain data as output

Systems do not coordinate each other. They do not know the “big picture”.

Examples:

- Match detection
- Gravity resolution
- Bomb resolution
- Cascading

System

System coordinate logic.

They:

- Own sequencing and orchestration
- Manage lifecycle and iteration
- Decide *when* systems run, not *how* they work

Services never contain core rules. They operate *on* logic, not *as* logic.

Advantages of This Architecture

This separation provides:

- Deterministic resolution from input to final state
- Clear ownership of responsibilities
- Testability at every layer
- No hidden side effects or event ordering issues
- Presentation fully decoupled from game logic

ENGINE LAYER

Swap Rules

- **ISwapRule**: Contract for validating a swap
(*input: SwapContext -> output: bool*)
- **AdjacentSwapRule**: Ensures two positions are adjacent
(*input: SwapContext -> output: bool*)
- **SwapContext**: Encapsulates swap evaluation data
(*position A, position B, isPlayerMove*)

Match Detection & Classification

- **IMatchDetector**: Detects raw matches on the board
(*input: BoardState -> output: MatchGroup[]*)
- **LineMatchDetector**: Finds horizontal and vertical line matches
(*input: BoardState -> output: MatchGroup[]*)

- **MatchGroup**: Contiguous group of matching gems
(positions, GemColor, MatchDirection)
- **MatchDirection**: Orientation of a match
(horizontal / vertical)
- **MatchClassifier**: Converts raw matches into semantic patterns
(input: MatchGroup[] -> output: ClassifiedMatch[])
- **ClassifiedMatch**: High-level match representation (L, T, 4-line, etc.)
(groups, MatchPattern)
- **MatchPattern**: Supported match shapes
(enum)

Bomb System

- **IBombCreationRule**: Defines how matches produce bombs
(input: ClassifiedMatch -> output: BombCreationResult)
- **BombCreationRule**: Concrete bomb creation logic
(input: ClassifiedMatch -> output: BombCreationResult)
- **BombCreationResult**: Describes bomb placement
(BombPlacement)
- **BombPlacement**: Where and how a bomb is placed
(Position, GemID)
- **BombTrigger**: Defines bomb activation
(BombID, BombPosition, AffectedGemID)
- **IBombResolver**: Resolves bomb effects on the board
(input: BoardState, bombPosition -> output: AffectedGems)
- **BombResolver**: Concrete bomb resolution implementation
(input: BoardState, bombPosition -> output: AffectedGems)

Board State & Spatial Data

- **BoardState**: Authoritative logical board state
(cells[,], width, height)

- **CellState**: State of a single board cell
(empty or GemState)
- **GemState**: Logical gem representation
(GemColor, GemType, GemId)
- **GemColor**: Matchable gem color
(enum)
- **GemType**: Gem behavior type
(normal, bomb)
- **Position**: Board coordinate
(x, y)
- **BoardSpatialMap**: Board layout in world coordinates
(World to Grid Coord, Grid to world coord)
- **BoardSnapshot**: Immutable snapshot for presentation
(derived from BoardState)

Cascading & Gravity

Note: Cascade in this context means board resolution loop until board is stable

- **ICascadeSystem**: Contract for cascading resolution
(input: BoardState -> output: CascadeIterationResult)
- **CascadeSystem**: Orchestrates repeated resolution until stable
(InitialBoardState -> FinalBoardState)
- **CascadeIterationResult**: Result of one cascade pass
- **CascadeStep**: Single cascade resolution pass record
(matches, bombs, gravity, spawned gems)
- **IGravitySystem**: Contract for gravity resolution
(input: BoardState -> output: ColumnGravityResolution[])
- **GravitySystem**: Applies gravity deterministically
(InitialBoardState -> FinalBoardState)

- **ColumnGravityResolution:** *Falling behavior for a column
(from -> to positions)*
- **ColumnSpawnResolution:** *Newly spawned gems in a column
(positions, GemState)*
- **IGemSpawner:** *Contract for gem generation
(InitialBoardState -> FinalBoardState)*
- **GemSpawner:** *Concrete gem spawning logic
(InitialBoardState -> FinalBoardState)*
- **GemIdGenerator:** *Generates unique gem identifiers
(output: GemId)*

Resolution Tracing

- **ResolutionTrace:** *Ordered record of all resolution steps
(for playback and debugging)*
- **ResolutionTraceBuilder:** *Incrementally builds resolution trace*

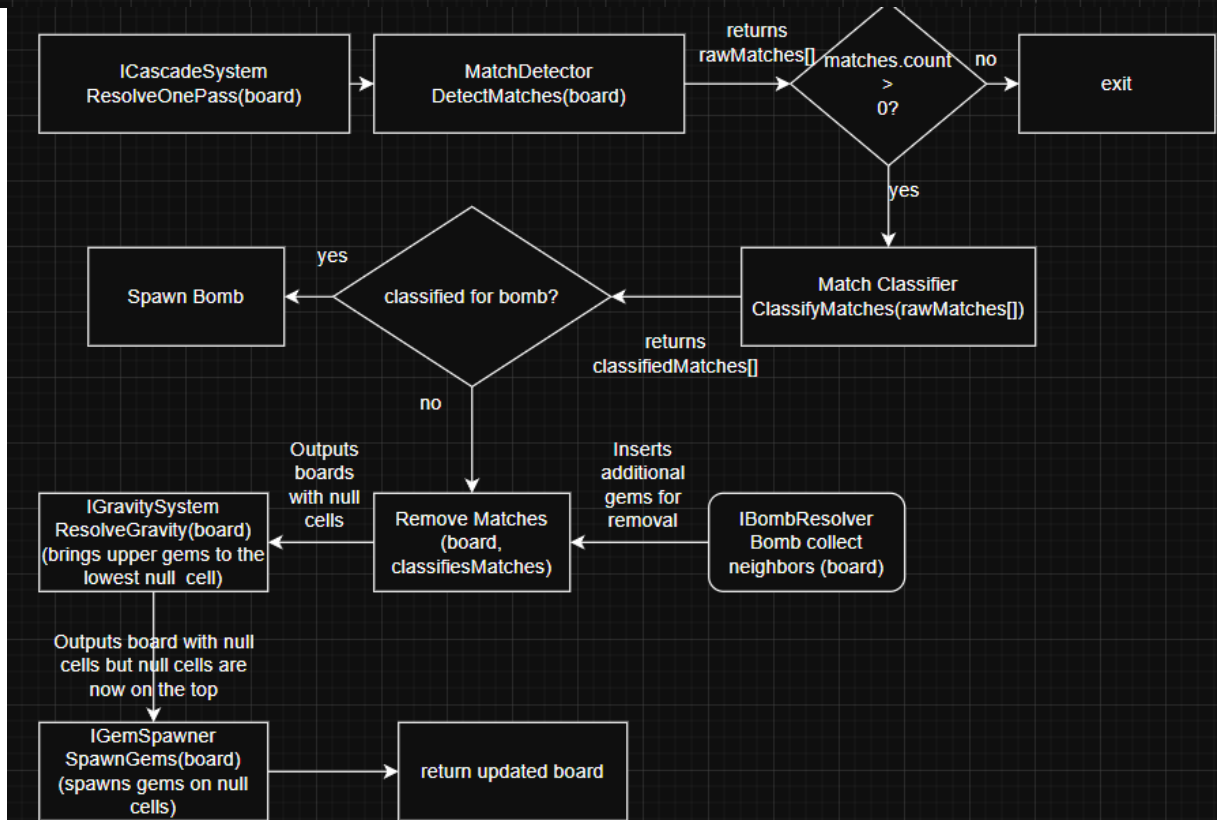
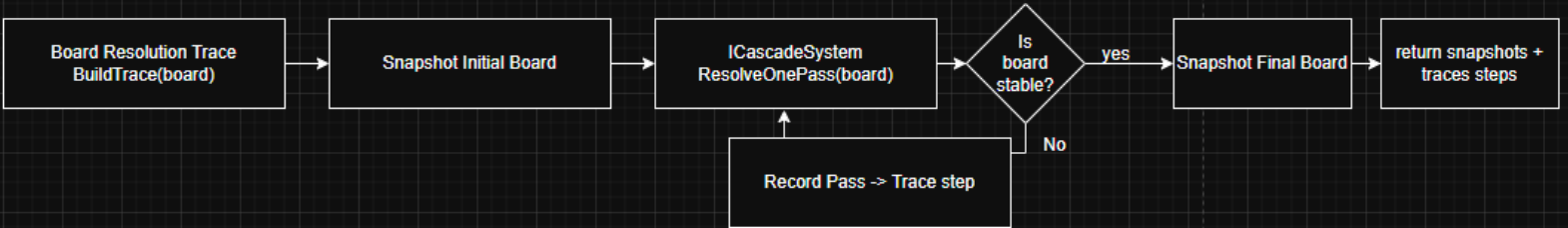
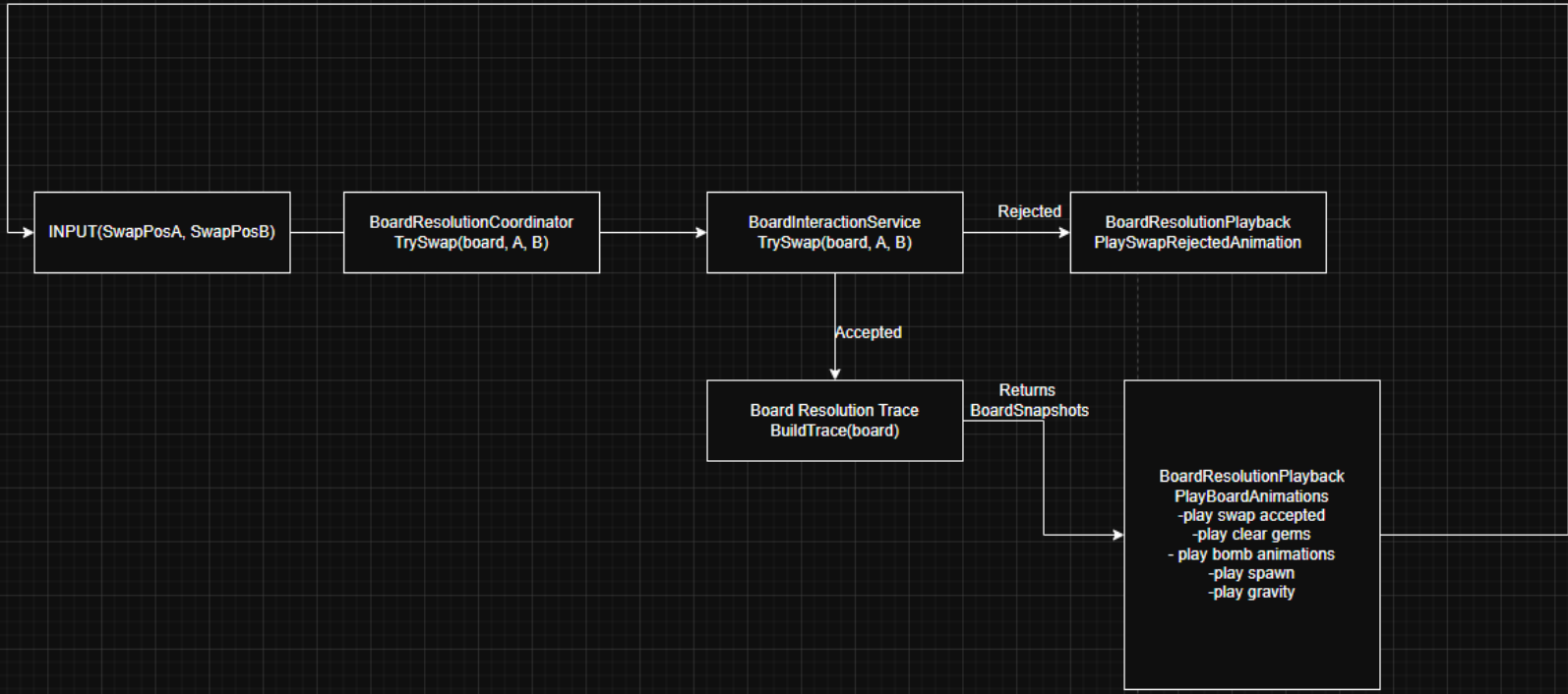
PRESENTATION

- **IBoardAnimation:** *Contract for board animations
(input: ResolutionTrace -> Single Animation for a cascade step)*
- **BoardView:** *Visual representation of the board*
- **CellView:** *Visual cell container*
- **GemView:** *Visual gem instance*
- **GemViewPool:** *Reuses gem visuals
(performance optimization)*
- **BoardResolutionPlayback:** *Plays board snapshots as animations
(input: ResolutionTrace)*

APPLICATION

- ***BoardResolutionCoordinator***: Coordinates input, engine, and playback
(input -> engine -> presentation)
- ***BoardInputController***: Converts user input into swap intents
(input -> Position, Position)

HIGH LEVEL FLOW:



TESTS:

- ▼ ✓ Domain
 - ▼ ✓ BoardStateTests
 - ✓ AccessingOutOfBoundsThrows
 - ✓ Board_CreatesCorrectDimensions
 - ✓ Board_InitializesAllCells
 - ✓ ClearGem_RemovesGem
 - ✓ Clone_CreatesDeepCopy
 - ✓ SetGem_PlacesGemCorrectly
 - ▼ ✓ Bomb
 - ▼ ✓ BombCreationRuleTests
 - ✓ TryCreate_BombColor_MatchesMatchColor
 - ✓ TryCreate_CascadeMatch_LOrTShape_DoesNotCreateBomb
 - ✓ TryCreate_CascadeMatch_ReturnsNull
 - ✓ TryCreate_PlayerMatch_LShape_CreatesBomb
 - ✓ TryCreate_PlayerMatch_SizeFour_CreatesBombAtSwapOrigin
 - ✓ TryCreate_PlayerMatch_SizeThree_ReturnsNull
 - ✓ TryCreate_PlayerMatch_TShape_CreatesBomb
 - ▼ ✓ BombResolverTests
 - ✓ Resolve_AlwaysIncludesCenter
 - ✓ Resolve_BombInCenter_ReturnsAll13Positions
 - ✓ Resolve_BombInCorner_ReturnsOnlyValidPositions
 - ✓ Resolve_BombNearEdge_ClipsOutOfBounds
 - ✓ Resolve_NoDuplicatePositions
 - ▼ ✓ CellStateTests
 - ✓ Cell_StartsEmpty
 - ✓ ClearGem_RemovesGem
 - ✓ Clone_CopiesGemButNotReference
 - ✓ SetGem_AssignsGem
 - ▼ ✓ GemStateTests
 - ✓ Clone_CreatesIndependentCopy
 - ✓ GemState_StoresColorAndType
 - ▼ ✓ Match
 - ▼ ✓ LineMatchDetectorTests
 - ✓ Detect_BreaksMatchOnDifferentColor
 - ✓ Detect_DoesNotReturnMatchesForTwoInLine
 - ✓ Detect_FindsHorizontalMatchOfThree
 - ✓ Detect_FindsMatchLongerThanThree
 - ✓ Detect_FindsVerticalMatchOfThree
 - ▼ ✓ MatchClassifierTests
 - ✓ Classify_LineMatch_RemainsLine
 - ✓ Classify_LShapeDetected
 - ✓ Classify_NonOverlappingMatches_RemainSeparate
 - ✓ Classify_OverlappingMatches_FormsTShape
 - ▼ ✓ PositionTests
 - ✓ Position_StoresCoordinatesCorrectly
 - ✓ Position_WithSameValues_IsEqualByValue
 - ▼ ✓ Swap
 - ▼ ✓ AdjacentSwapRuleTests
 - ✓ AdjacentSwap_WithNoMatch_IsRejected
 - ✓ DiagonalSwap_IsRejected
 - ✓ SwapThatCreatesMatch_IsAccepted
 - ✓ SwapThatProducesNoMatch_IsRejected
 - ✓ SwapValidation_DoesNotMutateOriginalBoard

